Mini-VLM: A Pure RWKV Vision-Language Model for Efficient Visual Question Answering

Oliver Petrovic
1008923042, oliver.petrovic@mail.utoronto.ca
https://github.com/Olipet24/mini-vlm

Introduction

Modern Vision-Language Models (VLMs) require billions of parameters and huge GPU resources, making them impossible to run on everyday or mobile hardware. Mini-VLM is a compact VLM for Visual Question Answering (VQA) constrained to a strict **100MB storage budget**, targeting edge deployment: given a 224×224 image and a free-form text question, the model outputs one of the 1000 most common VQA answers (Figure 1). Mapping raw pixels and free-form text jointly onto a correct answer requires learned, non-linear visual and linguistic representations that hand-coded rules cannot approximate (Goyal et al., 2017), which is why deep learning is appropriate here. Rather than shrinking a Transformer-based VLM post hoc, our core contribution is a *unified RWKV pipeline*: an RWKV spatial bridge and language core replace attention everywhere, giving linear-time (not quadratic) cost in the combined visual+text sequence length (Peng et al., 2023), instead of the linear-projector-into-attention approach used by connectors such as LLaVA (Liu et al., 2023). We compare this directly against a control baseline that keeps the same frozen encoder but restores standard self-attention, and evaluate both on the full VQA v2.0 pool, a held-out test split, and newly collected photographs never used in tuning.

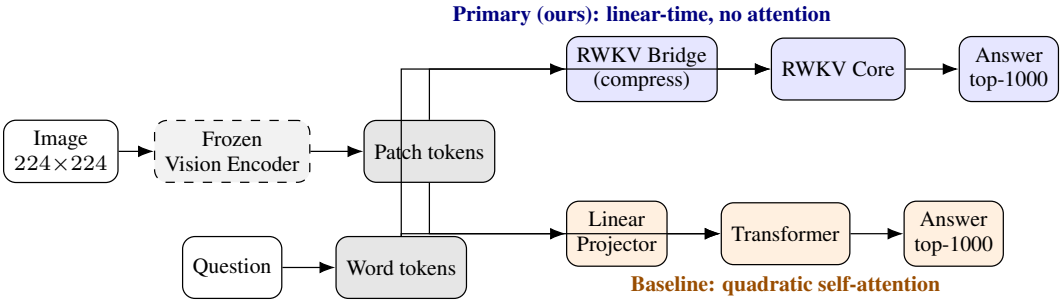

Figure 1: Both models share a frozen, offline-cached vision encoder and tokenizer; only the boxed bridge/projector and core/Transformer stages are trained. Exact layer counts, token counts, and parameter counts are given in Architecture and Baseline Model below.

Background & Related Work

Mini-VLM sits at the intersection of three lines of work. **Vision→language connectors.** LLaVA (Liu et al., 2023) projects every patch token through one linear layer straight into the language model, so sequence length – and attention cost – grows with image resolution. Flamingo's Perceiver Resampler (Alayrac et al., 2022) instead learns a small fixed set of query tokens that cross-attend into the visual features, compressing a variable-size feature map to a constant-size summary; our RWKV Spatial Bridge shares this compression goal but replaces the cross-attention itself with a learned linear pooling matrix, avoiding attention entirely. BLIP-2's Q-Former (Li et al., 2023) takes the cross-attention route further with a dedicated pretrained querying transformer – exactly the mechanism we avoid on compute-cost and parameter-budget grounds. **Efficient sequence models.** RWKV (Peng et al., 2023) replaces quadratic self-attention with a linear-time weighted-key-value recurrence; we use it as both the bridge and the language backbone, not just an efficient text decoder. **Compact deployment and VQA itself.** MobileNetV3 (Howard et al., 2019) targets exactly the mobile-inference regime we aim for and is our frozen backbone; GPTQ (Frantar et al., 2023) motivates our planned weight-compression path for the bridge/core's Linear layers. VQA v2.0 (Goyal et al., 2017) was built to

reduce language-prior bias in the original VQA dataset, but Agrawal et al. (2018) show such priors persist – motivating the question-only ablation below (37.94% accuracy from vision-blind inputs), which quantifies how much of our accuracy really comes from the image.

Data Processing

We use the official **VQA v2.0** dataset (questions/annotations over COCO images), cleaned in two steps to fit the 100MB budget: (1) collapse free-form answers to a **top-1000 most-common-answer classification** problem, built from the frequency distribution over the full 443,757-example train2014 pool (87.47% coverage), dropping examples outside that vocabulary; (2) resize/normalize images to 224×224 and pass them through the frozen vision encoder *once, offline*, caching the resulting $576 \times 7 \times 7$ FP16 feature map so training never re-runs the CNN. `train/val` are disjoint samples from the official *train2014* pool; `test` is sampled entirely from *val2014* – images and questions never seen during training or hyperparameter selection: 368,751 train, 19,407 val, 186,489 test questions over 82,575 (train+val) / 40,404 (test) unique images, 122,979 cached feature tensors, answer-type mix 43.1% yes/no : 43.5% other : 13.4% number, mean question length 6.1 words. One cleaned example: image of a bird, question "What color is this bird?", target class `white and brown` – one of the top-1000 classes covering 87.47% of the train2014 pool. Downloading and caching the full $\sim$123,000-image pool once was more reliable than smaller-scale per-image fetches, but not error-free: a batch of cached feature tensors was silently corrupted, spiking training loss until we added a feature-integrity check that now runs before every training job.

Architecture

The primary model's frozen **vision encoder** is MobileNetV3-Small, compressed FP32$\rightarrow$FP16 (3.66MB $\rightarrow$ 1.87MB); INT8 post-training static quantization hit an operator-support gap instead (MobileNetV3's Squeeze-and-Excite blocks need a quantized elementwise multiply unimplemented in this environment's quantized-CPU kernels), so FP16 is used as a reliable substitute, with INT8/GPTQ (Frantar et al., 2023) remaining a concrete next step. The **RWKV Spatial Bridge** – our main architectural contribution – flattens the encoder's $576 \times 7 \times 7$ feature map into 49 tokens, projects to $d_{model} = 128$, runs 2 RWKV blocks (RWKV time-mixing and squared-ReLU channel-mixing, implemented from scratch with a custom CUDA WKV kernel for GPU, numerically verified against a pure-Python reference implementation to a maximum output/gradient difference under 10^{-6}), then compresses to 16 language-aligned tokens via a *learned linear pooling matrix* (softmax-normalized weights, not cross-attention). The **RWKV Language Core** concatenates those compressed visual tokens with the question's word embeddings and passes the combined sequence through a 4-layer RWKV stack (dropout 0.2 throughout, weight decay 0.01 (default), selected by a validation-only hyperparameter search described under Baseline Model below), reading out with pooling mode last (default; `last_real` tested, did not help); the final hidden state feeds a linear classifier over 1000 answers. Total: **3,071,096 parameters**, 11.76MB state dict – combined with the 1.87MB FP16 vision encoder, a **13.63MB** deployed model, comfortably inside the 100MB budget.

Baseline Model

The baseline is the direct control group for isolating the RWKV design's contribution: it keeps the exact same frozen vision encoder, cached features, data splits, optimizer family, and random seed as the primary model, and differs only in how the two token streams are compressed and mixed – a plain **Linear Projector** maps all 49 spatial patch tokens (no compression) into a standard, unshared `nn.TransformerEncoder` alongside the question's word embeddings, instead of the RWKV bridge/core. This isolates the RWKV bridge and core as the one experimental variable, and at a comparable parameter count (2,574,696 vs. 3,071,096) it is a real architecture, not a strawman. **Reproducible specification:** $\mathrm{Linear}(576 \rightarrow 128)$ over all 49 flattened patch tokens; a learned positional embedding over $49 + T_q$ positions; word embeddings from the training-set tokenizer; `nn.TransformerEncoder`, 4 layers, 4 heads, $d_{model} = 128$, feedforward width 512, **pre-LN** (`norm_first=True`) – required because post-LN produced loss spikes and NaNs when training from scratch at $lr = 3 \times 10^{-3}$; a final `LayerNorm` on the readout token, then $\mathrm{Linear}(128 \rightarrow 1000)$; softmax cross-entropy loss. **Tuning protocol:** the baseline received the same six-configuration, validation-loss-selected hyperparameter search budget as the primary model (dropout, weight decay, depth, and learning rate) – addressing a specific weakness flagged in our progress report, where the

baseline used standard defaults untested against alternatives. The search's one clear winner was a lower learning rate (1×10^{-3} vs. the original 3×10^{-3}); dropout and weight decay stayed at their defaults (0.1 (default), 0.01 (default)) since no tested alternative beat them on validation loss. **Its own quantitative results: 47.43%** held-out test accuracy ($\pm$0.44% over 3 seeds, 2,574,696 params, 9.84MB), test loss 1.60, against a **21.84%** majority-class floor and a 37.94% question-only floor (Table 1); broken down by answer type, the baseline is strongest on yes/no (58.19%), then open-ended "other" (43.23%), and weakest on counting (29.84%). **Its own qualitative behaviour:** early in training an unshared Transformer trained from scratch was fragile – collapsing to one high-frequency answer regardless of question before the pre-LN fix – but the trained model now gives varied, often correctly confident answers, doing best on questions where its uncompressed 49 tokens preserve fine spatial detail (Figure 3).

Model	Params	Size	Test acc.	Test loss	Acc. y/n	Acc. other	Acc. num.
Majority-class floor	–	–	21.84%	–	–	–	–
Question-only (blind) floor	–	–	37.94%	–	–	–	–
Baseline (Linear + Transformer)	2,574,696	9.84MB	47.43%	1.60	58.19%	43.23%	29.84%
Primary (RWKV bridge + core)	3,071,096	11.76MB	45.76%	1.64	58.00%	39.45%	28.95%
New-data eval (n=TBD) – baseline	TBD (95% CI TBD)						
New-data eval (n=TBD) – primary	TBD (95% CI TBD)						

Table 1: Baseline vs. primary, full-scale test set (186,489 questions: 80,539 yes/no, 80,930 other, 25,020 number), each tuned via a 6-configuration validation-loss-selected search, 3-seed mean test accuracy, plus the newly collected photo evaluation (see Evaluate on New Data).

Quantitative Results

Overall test accuracy is close between the two models (47.43% vs. 45.76%, 3-seed means), so we look past the headline number: the per-type breakdown (Table 1) shows the two are essentially tied on yes/no (58.19% vs. 58.00%) but the baseline is clearly ahead on open-ended "other" (43.23% vs. 39.45%), where its uncompressed 49 tokens and full self-attention retain fine-grained detail the bridge's pooling discards; a sweep over the bridge's compressed-token count K (Figure 2, right) shows accuracy rising from K=4 to K=16, confirming this is a real compression bottleneck rather than noise, even though the largest K we tried still trails the baseline. Both models are weakest on counting (29.84% vs. 28.95%) – a limitation that persists regardless of architecture, suggesting it is a property of the task and representation rather than something either compression scheme specifically causes. The question-only blind floor (37.94%) quantifies how much of either model's accuracy is attributable to language priors alone, independent of the image (Agrawal et al., 2018).

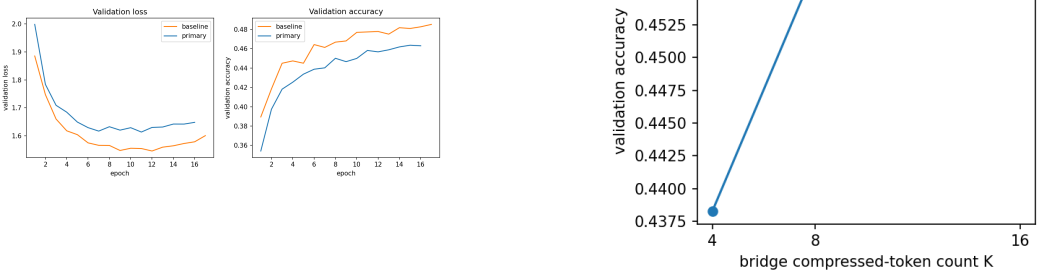

Figure 2: Left: baseline vs. primary validation loss/accuracy across training. Right: primary test accuracy as the bridge's compressed-token count K varies, isolating the 49$\rightarrow K$ bottleneck's effect on accuracy.

Qualitative Results

Figure 3 shows four hand-selected held-out examples chosen to cover the interesting cases, not a random sample: one both models answer correctly, one only primary gets right, one only baseline gets right, and one both get wrong. When primary is confident on a yes/no question it tends to be right; on harder "other" questions it can be confidently wrong, consistent with a frozen encoder and

a handful of compressed tokens losing detail needed to localize a specific object. The baseline's characteristic failure mode is the opposite: lower peak confidence but a more even hit rate across open-ended questions, matching its retained fine-grained detail.

Figure 3: Four held-out test questions with both models' top prediction and confidence overlaid, chosen per the selection rule above (correct predictions in green, incorrect in red).

Evaluate on New Data

We evaluate generalization on two disjoint tiers of held-out data. **Tier 1**, the 186,489-question COCO test split, is drawn entirely from *val2014*, disjoint from the train2014 pool used for training/validation and – mechanically enforced by a `-skip-test` flag on every search run – never touched during model selection. **Tier 2** is $n = TBD$ newly photographed images and hand-written questions we collected and labelled after every hyperparameter was frozen, evaluated exactly once: baseline TBD (95% CI TBD), primary TBD (95% CI TBD), vs. a TBD majority floor on this new set (Table 1). [Tier 1→2 gap discussion: TBD once Tier 2 results land.]

Discussion

At **45.76%** test accuracy against a 21.84% majority-class floor and a 37.94% question-only floor, both models answer real visual questions well above chance at $\leq$13.63MB, comfortably under budget. Two results surprised us, and we report both honestly rather than only the flattering half. First, the architecturally "smarter" RWKV bridge did not beat the linear-projector baseline – the K-sweep (Figure 2) confirms a real compression bottleneck (accuracy rises from K=4 to 16), but even our largest tested K still trails. Second, and less expected: our validation-loss-selected search (run at 12 epochs, single seed, for wall-clock reasons) picked dropout=0.2 and K=16 for primary, yet the final 3-seed retrain came in *below* our original untuned primary (45.76% vs. 46.70%), while the equivalently-tuned baseline barely moved (47.43% vs. 47.38%). A short, single-seed validation proxy evidently did not transfer to the longer, multi-seed training regime it was meant to select for – a concrete lesson about the limits of a small hyperparameter search budget, not a result we would have predicted going in. On raw wall-clock efficiency, our custom CUDA WKV kernel (median 3.4ms at T_q=16) did not beat PyTorch's built-in attention (1.3ms) at this project's actual token-count scale ($\sim$50-100 tokens per example) – RWKV's linear-time advantage is real asymptotically, but only becomes a wall-clock win at sequence lengths beyond what a compressed-token pipeline like this one uses; again, we report this rather than overclaim an unmeasured speed benefit. Enforcing validation-only selection (`-skip-test`) throughout is what makes our final numbers mean something rather than being quietly tuned on the test set. Limitations: the closed vocabulary cannot answer 87.47% of the training pool itself; there is no autoregressive decoding; the frozen encoder likely bounds the new-data gap; and INT8/GPTQ remains untried.

Ethical Considerations

Our blind ablation (37.94% accuracy with the image zeroed out) quantifies how much either model answers from linguistic priors alone (Agrawal et al., 2018), not a hypothetical failure mode. A $\leq$13.63MB on-device VQA model's clearest use case is assistance for blind and low-vision users, where a *confidently wrong* answer is worse than an abstention; even when wrong, both models average 0.45–0.48 top-1 confidence (vs. 0.61–0.63 when right) on a 1000-way problem, high enough to argue for an abstention threshold before any real deployment. COCO's own imagery skew, plus a constrained model falling back on priors, means performance should not be assumed to generalize evenly – a concern our own photo set was designed to probe. We used only licensed COCO imagery and photographs we personally captured with consent.

References

Goyal, Y., Khot, T., Summers-Stay, D., Batra, D., & Parikh, D. (2017). Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering. *CVPR*.

Peng, B., et al. (2023). RWKV: Reinventing RNNs for the Transformer Era. *EMNLP*.

Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual Instruction Tuning. *NeurIPS*.

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *ICLR*.

Alayrac, J.-B., et al. (2022). Flamingo: a Visual Language Model for Few-Shot Learning. *NeurIPS*.

Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. *ICML*.

Howard, A., et al. (2019). Searching for MobileNetV3. *ICCV*.

Agrawal, A., Batra, D., Parikh, D., & Kembhavi, A. (2018). Don't Just Assume; Look and Answer: Overcoming Priors for Visual Question Answering. *CVPR*.